

CLAUDE CODE: FROM FIRST INSTALL TO LIFE AFTER PRODUCTION

The Working Day

Claude Code deep dive, episode 9

Claude stops when the work LOOKS done. So you became the verification loop.

One sentence in the documentation explains more about the difference between a good day and a bad one than anything else in it: Claude stops when the work looks done.

Without a check it can run, "looks done" is the only signal available, and every mistake waits for you to notice it. This sheet is how to fix that, plus the failure patterns you are probably already in.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- Giving it a check it can run
- The four levels of how hard that check gates the work
- Evidence rather than assertions of success
- Explore, plan, implement, commit, and when to skip the plan
- Letting it interview you before a big feature
- When to reach for a hook
- How to describe a bug
- The documented failure patterns

Out of scope

- CI configuration in detail
- Deployment tooling, which is yours and not this tool's
- Any claim about how much faster this makes anyone
- Prompt "tricks": none of this is about phrasing

Almost everything on this sheet is actionable today, with no setup, which is unusual for a page about an agentic tool.

01 The four levels of verification

In the prompt	works today, no setup
A goal condition	re-checked after every turn
A Stop hook	blocks the turn from ending
A verification subagent	a different grader
Stop hook override	after 8 consecutive blocks
Corrections before you /clear	TWO

These are interview assumptions chosen to make the behaviour concrete. They are not published figures from any provider and no claim is made about any particular deployment.



03 Give it something that returns pass or fail

Claude stops when the work looks done. Give it a check it can run and the loop closes on its own: it does the work, runs the check, reads the result, and iterates until the check passes.

A check is anything that returns a signal it can read. A test suite. A build exit code. A linter. A script that diffs output against a fixture. A browser screenshot compared against the design.

The documentation's own summary is the best one: it is the difference between a session you watch and one you walk away from.

Claude stops when the work looks done. Without a check it can run, "looks done" is the only signal available, and you become the verification loop.

Claude Code docs, Best practices, checked September 2026

Give Claude a check it can run: tests, a build, a screenshot to compare. It is the difference between a session you watch and one you walk away from.

Claude Code docs, Best practices, checked September 2026

04 Evidence, not assertions

Have it show the test output, the command it ran and what came back, or a screenshot of the result.

Reviewing evidence is faster than re-running the verification yourself, and it works for sessions you were not watching.

Put more bluntly: done is a claim, output is a fact. If all you get back is the word done, you have not saved verification work, you have moved it to later when you have forgotten the context.

Have Claude show evidence rather than asserting success: the test output, the command it ran and what it returned, or a screenshot of the result.

Claude Code docs, Best practices, checked September 2026

05 Go and look: /diff

Evidence works when the agent cooperates. /diff is how you check without needing it to. It shows the changes in your working tree without leaving the session: what Claude has edited so far, alongside anything else you had not committed.

In a wide enough terminal it opens as a panel beside the conversation and stays there, refreshing every time a file is edited or a command runs.

THE PART THAT CATCHES PEOPLE IS WHAT THE LIST LEAVES OUT. By default it skips test files, skips generated files, and folds every change from before this session into a single line at the bottom. All three expand with a click. The risk is not that they are hidden, it is that you did not know to look, and a review that silently omits the tests is the exact failure this sheet argues against.

Ctrl+X B cycles what the panel compares against: this session's changes, then everything you have not committed, then everything since your branch split from the default branch.

Run /diff to look over the changes in your working tree without leaving Claude Code, including the edits Claude has made so far alongside anything else you have not committed.

Claude Code docs, Interactive mode, checked September 2026

The diff panel list skips test files and generated files, and collapses changes from before this session into one line at the bottom; click either count line to expand it.

Claude Code docs, Interactive mode, checked September 2026

The panel needs fullscreen rendering, a git repository, and a terminal at least 110 columns wide.

Claude Code docs, Interactive mode, checked September 2026

06 Planning has a cost, and the documentation says so

The shape is explore, plan, implement, commit. Explore in plan mode, ask for a plan, edit the plan yourself if you want, then implement against it.

But plan mode adds overhead. For tasks where the scope is clear and the fix is small, ask it to do the work directly.

The test is beautifully concrete: IF YOU COULD DESCRIBE THE DIFF IN ONE SENTENCE, SKIP THE PLAN. Planning is most useful when you are uncertain about the approach, when the change touches multiple files, or when you are unfamiliar with the code.

Plan mode is useful but adds overhead. For tasks where the scope is clear and the fix is small, ask Claude to do it directly. If you could describe the diff in one sentence, skip the plan.

Claude Code docs, Best practices, checked September 2026



07 Do not write the spec

For larger features, have Claude interview you first. Ask it to cover technical implementation, interface, edge cases, concerns and tradeoffs, to skip the obvious questions and dig into the hard parts, and to keep going until everything is covered, then write the spec to a file.

Then start a FRESH session to execute it. The new session has clean context aimed entirely at implementation, and you have a written spec to reference.

The documentation's note on what makes a spec good is worth memorising: name the files and interfaces involved, state what is out of scope, and end with an end-to-end verification step.

For larger features, have Claude interview you first, then start a fresh session to execute the resulting spec.

Claude Code docs, Best practices, checked September 2026

The most useful specs are self-contained: they name the files and interfaces involved, state what is out of scope, and end with an end-to-end verification step.

Claude Code docs, Best practices, checked September 2026

08 When to reach for a hook

Unlike CLAUDE.md instructions, which are advisory, hooks are deterministic and guarantee the action happens.

That is the whole decision. If it should usually happen, write it in the file. If it must happen every time with zero exceptions, it is a hook. Format after every edit. Lint before a commit. Block writes to a folder.

And you do not have to hand-write them: ask for a hook that runs your linter after every file edit. One caution: hooks run with your credentials and your environment, so treat one you did not write like any other dependency.

Unlike CLAUDE.md instructions which are advisory, hooks are deterministic and guarantee the action happens.

Claude Code docs, Best practices, checked September 2026

Hook handlers run in the current directory with Claude Code's environment, which includes the credentials available to that session.

Claude Code docs, Hooks, checked September 2026

09 How to describe a bug

Not "fix the login bug". Instead: users report that login fails after session timeout, check the auth flow in this directory, especially token refresh, write a failing test that reproduces the issue, then fix it.

Three things happened there. You gave the SYMPTOM rather than your guess at the cause. You pointed at where to look without insisting. And you asked for a reproduction before a fix, so the fix arrives with evidence that it worked.

There is a matching instruction for builds worth stealing verbatim: address the root cause, do not suppress the error. Without it, a failing type check has an obvious and terrible solution available.

Describe the symptom, the likely location, and what "fixed" looks like, and ask for a failing test that reproduces the issue before the fix.

Claude Code docs, Best practices, checked September 2026

Ask Claude to address the root cause rather than suppress the error.

Claude Code docs, Best practices, checked September 2026

10 The documented failure patterns

The kitchen sink session: one task, then something unrelated, then back again, and the context is full of irrelevance. Clear between unrelated tasks.

Correcting over and over: after TWO failed corrections on the same issue, stop. Clear and write a better prompt incorporating what you learned. A clean session with a better prompt almost always beats a long session with accumulated corrections.

The over-specified CLAUDE.md: if it is too long, important rules get lost. For each line ask whether removing it would cause a mistake. And if you emphasise many lines, none of them stands out.

After two failed corrections on the same issue, run /clear and start fresh with a more specific prompt that incorporates what you learned.

Claude Code docs, Best practices, checked September 2026

Bloated CLAUDE.md files cause Claude to ignore your actual instructions. If you emphasize many lines, none of them stands out.

Claude Code docs, Best practices, checked September 2026

11 The most honest line in the documentation

Before treating a task as done, a subagent can review the diff in a fresh context, seeing only the change and your criteria rather than the reasoning that produced it.

And then the caveat, which is unusual for a vendor to print about its own recommended technique: a reviewer prompted to find gaps will usually report some, even when the work is sound, because that is what it was asked to do.

Chasing every finding leads to over-engineering: extra abstraction layers, defensive code, and tests for cases that cannot happen. Tell the reviewer to flag only gaps that affect correctness or the stated requirements, and treat the rest as optional.

A reviewer prompted to find gaps will usually report some, even when the work is sound, because that is what it was asked to do. Chasing every finding leads to over-engineering.

Claude Code docs, Best practices, checked September 2026

A reviewer running in a fresh subagent context sees only the diff and the criteria you give it, not the reasoning that produced the change.

Claude Code docs, Best practices, checked September 2026

The working day, as a checklist

1. Does this task have a check it can run?
2. If it runs unattended, is the check a goal or a Stop hook?
3. Are you getting evidence, or the word "done"?
4. Could you describe the diff in one sentence? Then skip the plan
5. Is this big enough that it should interview you first?
6. Is this rule advisory, or must it happen every time? That decides hook or file
7. Did you describe the symptom, or your guess at the cause?
8. Have you corrected the same thing twice? Then clear and rewrite



Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

Facebook

<https://www.facebook.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

☕ Buy me a coffee

<https://buymeacoffee.com/codewithsam>

Buy me sadza

<https://buymeacoffee.com/codewithsam>

Best practices

<https://code.claude.com/docs/en/best-practices>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.